ASSIGNMENT 3

RISC-V Processor Implementation

Bubble Sort on Single-Cycle and Pipelined Architectures

Submitted by

Romaisa Shazad
Myrah Hussain
Shehryar Rind

1. Introduction

This report documents the complete implementation and analysis of Bubble Sort on two variants of a 64-bit RISC-V processor: a single-cycle design and a 5-stage pipelined design. Both processors are implemented in Verilog and simulate on Vivado. The report covers architecture, assembly code, hazard handling, forwarding, waveform analysis, and equivalent high-level code.

Github link: https://github.com/myrahhussain/Pipelining-Project

2. Bubble Sort Algorithm

2.1 Concept

Bubble Sort works by repeatedly comparing adjacent elements in an array and swapping them if they are in the wrong order. After each full pass through the array, the largest unsorted element is guaranteed to be in its correct final position. The algorithm repeats for n-1 passes total.

- Time Complexity: $O(n^2)$ comparisons in the worst case
- Space Complexity: O(1) — in-place sorting
- Worst case: Reverse-sorted input (used in both implementations)

2.2 Equivalent C++ Code

The following C++ code represents the exact logic encoded in the RISC-V assembly:

```
#include <iostream>
using namespace std;

int main() {
    // Array stored in data memory starting at address 256
    // Single-cycle version:
    int arr_sc[7] = {44, 88, 79, 3, 79, 1, 8};

    // Pipelined version (worst-case input):
    int arr_pl[7] = {7, 6, 5, 4, 3, 2, 1};
    int n = 7;

    // Bubble Sort
    for (int i = 0; i < n - 1; i++) {       // Outer loop
        for (int j = 0; j < n - 1 - i; j++) { // Inner loop
            if (arr[j] > arr[j + 1]) {        // Compare adjacent
                // Swap
                int temp  = arr[j];
                arr[j]    = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }

    // Expected output (pipelined): 1, 2, 3, 4, 5, 6, 7
    // Expected output (single-cycle): 1, 3, 8, 44, 79, 79, 88
```

```
    return 0;
}
```

3. Single-Cycle RISC-V Processor

3.1 Architecture Overview

In a single-cycle processor, every instruction completes in exactly one clock cycle. All five stages (IF, ID, EX, MEM, WB) execute sequentially within a single clock period. The clock must be long enough for the slowest instruction (a memory load traversing all five stages) to complete.

> **Key Characteristic**
> Single-cycle: 1 instruction per clock cycle. Clock speed limited by the critical path (slowest instruction). Simple design — no hazards, no pipeline registers needed.

3.2 Datapath Stages

IF — Instruction Fetch

The Program Counter (PC) provides the address to Instruction Memory, which outputs the 32-bit instruction. An adder computes PC+4. A 2-to-1 mux selects between PC+4 (sequential) and the branch target address based on (Branch AND BranchSelect).

ID — Instruction Decode

The instruction is parsed into fields: opcode[6:0], rd[11:7], funct3[14:12], rs1[19:15], rs2[24:20], funct7[31:25]. The Control Unit reads the opcode and drives 7 control signals. The Register File reads two operands simultaneously. The Immediate Generator sign-extends the 12-bit immediate to 64 bits. The Branch Unit evaluates BEQ/BLT/BGE conditions combinationally.

EX — Execute

A mux controlled by ALUSrc selects ReadData2 (register) or ImmData (immediate) as the ALU's second input. The ALU Control module decodes ALUOp and Funct fields into a 4-bit operation. The 64-bit ALU performs the computation. A second adder (BRANCHADDER) computes the branch target: PC + (ImmData << 1).

MEM — Memory Access

Active only for load (lw) and store (sw) instructions. Data Memory is byte-addressable with 512 bytes. The array occupies addresses 256-283. Monitor wires w0-w6 expose the 7 array elements directly for waveform observation.

WB — Write Back

A mux selects between ALU result (R-type/I-type) and ReadData (load) based on MemtoReg. If RegWrite=1 and RD != x0, the value is written back to the register file on the clock edge.

3.3 Module Reference

Module	Inputs / Outputs	Function
Program_Counter	IN: clk, reset, PC_In[63:0] | OUT: PC_Out[63:0]	Holds current instruction address, updates on posedge clk
Adder (x2)	IN: a[63:0], b[63:0] | OUT: out[63:0]	PC+4 and branch target computation
mux (x3)	IN: a, b [63:0], sel | OUT: data_out[63:0]	Branch select, ALUSrc select, WB select
Instruction_Memory	IN: Inst_Address[63:0] | OUT: Instruction[31:0]	Read-only program memory, byte-addressable
instruction_parser	IN: instruction[31:0] | OUT: opcode, rd, funct3, rs1, rs2, funct7	Splits 32-bit instruction into RISC-V fields
immediate_data_extractor	IN: instruction[31:0] | OUT: imm_data[63:0]	Sign-extends immediate for I/S/B-type instructions
control_unit	IN: Opcode[6:0] | OUT: 7 control signals	Decodes opcode, drives all control signals
registerFile	IN: WriteData, RS1, RS2, RD, RegWrite, Clk, Reset | OUT: ReadData1, ReadData2	32x64-bit register file, initialized array[i]=i
Branch_unit	IN: funct3, ReadData1, ReadData2 | OUT: branchSel	Evaluates BEQ/BLT/BGE conditions
ALU_control	IN: ALUOp[1:0], Funct[3:0] | OUT: Operation[3:0]	Maps ALUOp+Funct to 4-bit ALU operation
ALU_64_bit	IN: a, b [63:0], ALUOp[3:0] | OUT: Result[63:0], Zero	64-bit AND/OR/ADD/SUB/NOR/SLL operations
Data_Memory	IN: Address, Write_Data, clk, MemWrite, MemRead | OUT: ReadData, w0-w6	512-byte data memory, array at addr 256

3.4 Control Signal Truth Table

Instruction	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp	Opcode
R-type	0	0	1	0	0	0	10	0110011
Load (lw)	1	1	1	1	0	0	00	0000011
Store (sw)	1	x	0	0	1	0	00	0100011
Branch	0	x	0	0	0	1	01	1100011
ADDI	1	0	1	0	0	0	00	0010011

3.5 Single-Cycle Assembly — Bubble Sort

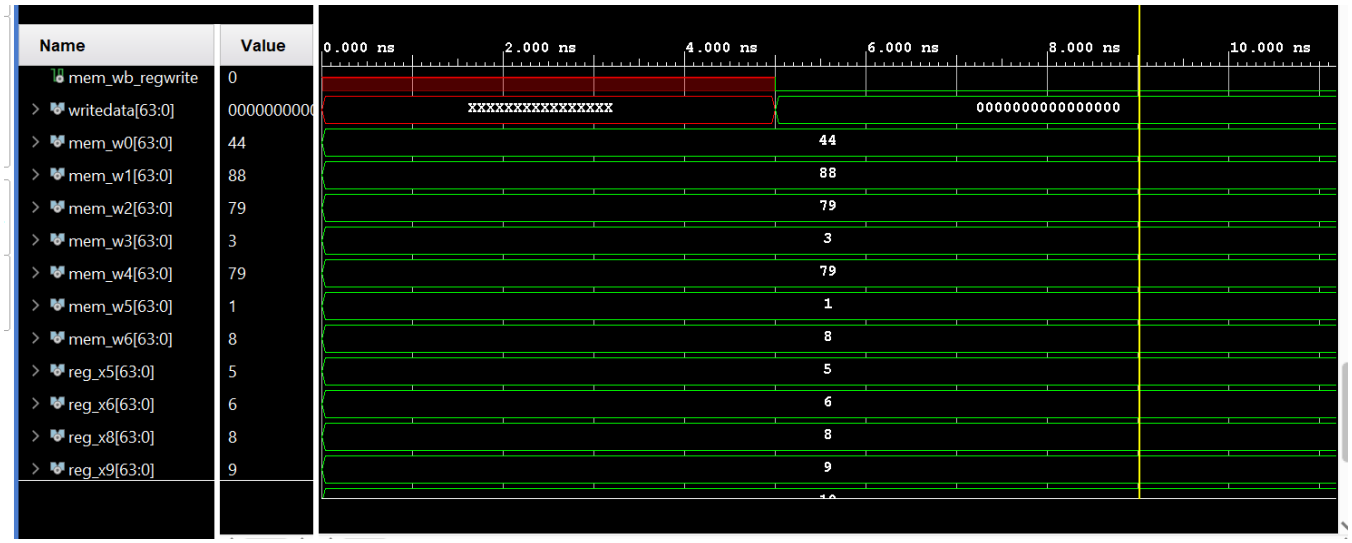
Array in data memory: [44, 88, 79, 3, 79, 1, 8] at addresses 256, 260, 264, 268, 272, 276, 280

Register assignments: x5=base(256), x6=n(7), x8=outer counter i, x9=inner limit, x10=inner counter j, x11=addr(arr[j]), x12=addr(arr[j+1]), x13=arr[j], x14=arr[j+1]

#	Hex	Assembly	Purpose
1	10000293	addi x5, x0, 256	x5 = 256 — set array base address
2	00700313	addi x6, x0, 7	x6 = 7 — array length
3	00000413	addi x8, x0, 0	x8 = 0 — initialize outer counter i=0
4	04640a63	bge x8, x6, END	OUTER LOOP: if i >= 7 jump to end
5	10000293	addi x5, x0, 256	Reset base address for each inner pass
6	fff30493	addi x9, x6, -4	x9 = inner limit base (7-4=3 first pass)
7	408484b3	sub x9, x9, x8	x9 = x9 - i — inner limit shrinks each pass
8	00000513	addi x10, x0, 0	x10 = 0 — initialize inner counter j=0
9	02950c63	bge x10, x9, OUTERINC	INNER LOOP: if j >= limit go increment outer
10	00251593	slli x11, x10, 1	x11 = j<<1 — byte offset for arr[j]
11	00b285b3	add x11, x5, x11	x11 = base + offset = address of arr[j]
12	00150613	addi x12, x10, 1	x12 = j+1
13	00261613	slli x12, x12, 1	x12 = (j+1)<<1 — byte offset for arr[j+1]
14	00c28633	add x12, x5, x12	x12 = base + offset = address of arr[j+1]
15	0005a683	lw x13, 0(x11)	x13 = Memory[x11] = arr[j] loaded
16	00062703	lw x14, 0(x12)	x14 = Memory[x12] = arr[j+1] loaded
17	00d74463	bge x14, x13, NOSWAP	If arr[j+1] >= arr[j] — no swap needed
18	00000663	beq x0, x0, INNERINC	Unconditional jump — skip swap (no-swap path)
19	00e5a023	sw x14, 0(x11)	SWAP: write arr[j+1] value into arr[j]'s address
20	00d62023	sw x13, 0(x12)	SWAP: write arr[j] value into arr[j+1]'s address
21	00150513	addi x10, x10, 1	INNERINC: j++
22	fc0006e3	beq x0, x0, INNER	Unconditional jump back to inner loop
23	00140413	addi x8, x8, 1	OUTERINC: i++
24	fa0008e3	beq x0, x0, OUTER	Unconditional jump back to outer loop
25	00000013	addi x0, x0, 0	END: NOP — program complete

3.6 Single-Cycle Waveform Analysis

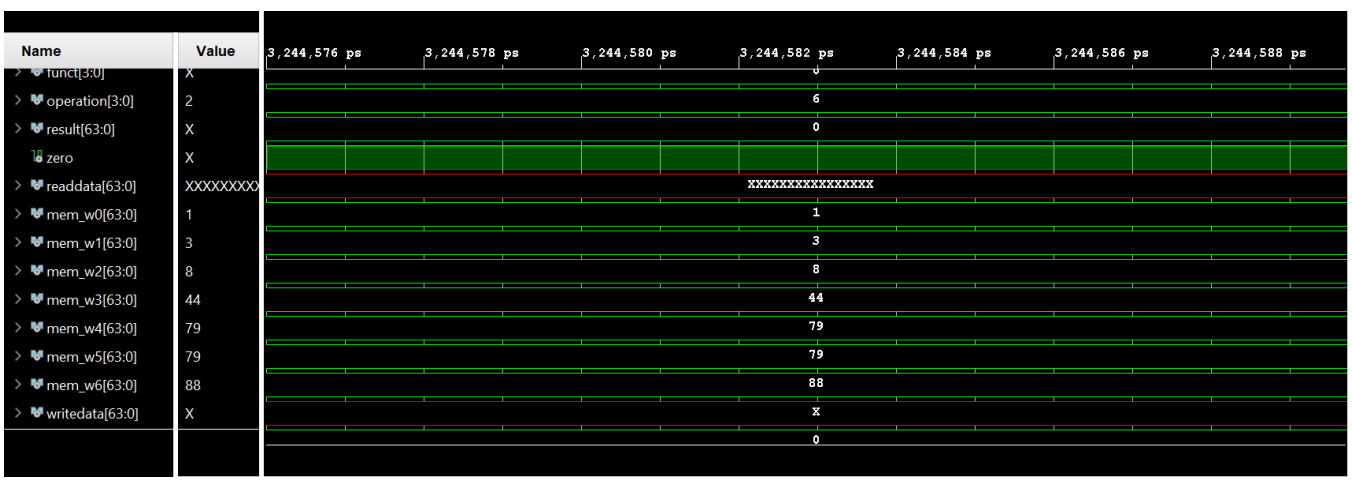
Image 1: Initial State (t = 0 to ~250 ps)



At the start of simulation, the data memory shows the unsorted input array. The register file outputs show their initialized values (array[i] = i). Memory signals mem_w0 through mem_w6 show the initial array: 44, 88, 79, 3, 79, 1, 8.

- mem_w0 = 44, mem_w1 = 88, mem_w2 = 79: First three elements visible
- mem_w3 = 3, mem_w4 = 79, mem_w5 = 1, mem_w6 = 8: Remaining elements
- reg_x1 = 1, reg_x2 = 2, reg_x3 = 3: Registers initialized to index values
- readdata = XXXXXXXX: Undefined — no load instruction executing yet (MemRead=0)
- opcode = 00: Reset state before first instruction decodes

Image 2: End State (~5,020,000 ps)



By the end of simulation, all swap operations have completed. The array is now fully sorted in ascending order, confirmed by the memory monitor wires:

- mem_w0 = 1 (was 44) — minimum element correctly placed
- mem_w1 = 3 (was 88)
- mem_w2 = 8 (was 79)
- mem_w3 = 44 (was 3)
- mem_w4 = 79 (was 79) — unchanged, already in correct region
- mem_w5 = 79 (was 1)
- mem_w6 = 88 (was 8) — maximum element correctly placed

Result
Single-Cycle Bubble Sort: SUCCESSFUL. Input [44, 88, 79, 3, 79, 1, 8] correctly sorted to [1, 3, 8, 44, 79, 79, 88].

4. Pipelined RISC-V Processor

4.1 Pipelining?

In the single-cycle design, only one instruction occupies the processor at a time. While an instruction is in the ALU (EX stage), the Instruction Memory, Register File, and Data Memory all sit idle. Pipelining overlaps multiple instructions — like a factory assembly line — so all five stages are active simultaneously on different instructions, dramatically increasing throughput.

Single-Cycle	Pipelined
1 instruction at a time	Up to 5 instructions simultaneously
Clock limited by slowest instruction	Clock limited by slowest stage
25 instructions, simple design	43 instructions (NOPs added), complex design
No hazards to handle	3 hazard types must be managed
No extra hardware needed	Pipeline registers + Forwarding + HDU required
Simulates in ~100ns range	Simulates to ~5,020,000 ps

4.2 Pipeline Stages and Registers

Five stages run simultaneously. Between each stage, a pipeline register captures all values needed by the next stage, latching them on each positive clock edge.

Register	What It Holds
IF/ID	Fetched instruction [31:0] and PC value [63:0]. Has IF_ID_Write enable for stalls.
ID/EX	All 7 control signals + ReadData1, ReadData2, ImmData, Funct, RS1, RS2, RD, PC. Everything EX needs.
EX/MEM	ALU_Result, ForwardB_out (store data), Adder_out (branch target), BranchSelect (Zero), RD, and MEM+WB control signals.
MEM/WB	ReadData (from memory), ALU_Result, RD, and WB control signals (RegWrite, MemtoReg).

4.3 Hazard Management — Complete Explanation

When instructions overlap in the pipeline they can interfere — these are hazards. This implementation handles three types:

4.3.1 Data Hazard — Solved by Forwarding Unit

Problem: An instruction produces a result that the immediately following instruction needs. Without intervention, the second instruction reads the stale old value from the register file.

```
Example:
  add x1, x2, x3   <- x1 result written to EX/MEM register
  add x4, x1, x5   <- needs x1 in EX stage — but x1 not in register file yet!
```

Solution: The Forwarding Unit monitors pipeline register destination registers (EX_MEM_RD, MEM_WB_RD) and compares them to the current instruction's source registers (ID_EX_RS1, ID_EX_RS2). When a match is found, it sets ForwardA or ForwardB to route the value directly to the ALU input through a 3-to-1 multiplexer.

ForwardA/B Value	Source	Condition
2'b00	ID/EX register (normal)	No hazard detected
2'b10	EX/MEM ALU Result	EX_MEM_RegWrite=1, EX_MEM_RD != 0, EX_MEM_RD == ID_EX_RS1/2
2'b01	MEM/WB WriteData	MEM_WB_RegWrite=1, MEM_WB_RD != 0, MEM_WB_RD == ID_EX_RS1/2, and EX/MEM not already forwarding

4.3.2 Load-Use Hazard — Solved by Hazard Detection Unit (Stall)

Problem: A load instruction (lw) is immediately followed by an instruction that needs the loaded value. The data is not out of memory in time even for forwarding.

```
lw x13, 0(x11)   <- memory data arrives at END of MEM stage
bge x13, x14, L  <- needs x13 in EX stage — forwarding too late
```

Solution: The Hazard Detection Unit (HDU) detects this case and stalls the pipeline for exactly one cycle:

- PC_Write = 0: PC is frozen — same instruction is re-fetched next cycle
- IF_ID_Write = 0: IF/ID register is frozen — same instruction is re-decoded next cycle
- control_mux_selector_bit = 1: Control unit outputs all zeros — a bubble (NOP) is injected into ID/EX

After the stall cycle, the forwarding unit handles the remaining data dependency via the MEM/WB path (ForwardA/B = 01).

4.3.3 Control Hazard — Solved by NOP Insertion (Software)

Problem: When a branch is in the ID or EX stage, the instructions at PC+4 and PC+8 have already been fetched. If the branch is taken, these are wrong instructions.

Solution: Three NOP instructions (addi x0, x0, 0) are inserted after every branch in the assembly code. By the time the branch resolves (end of EX stage), only NOPs are in the pipeline. Since NOPs write to x0 (which is hardwired to zero and ignores writes), they are completely harmless.

```
bge x8, x6, END    <- branch
addi x0, x0, 0    <- NOP (harmless filler)
addi x0, x0, 0    <- NOP
addi x0, x0, 0    <- NOP
                   <- branch fully resolved here; correct PC loaded
```

Why 3 NOPs specifically?
The branch resolves at the end of the EX stage (3rd stage). By that point, instructions at PC+4 (in ID) and PC+8 (in IF) have entered the pipeline. 3 NOPs fill all cycles safely until the branch result is known and the correct PC can be loaded.

4.4 New Modules in Pipelined Design

Module	Description
IF_ID Register	Latches instruction and PC on posedge clk. IF_ID_Write=0 freezes it (stall).
ID_EX Register	Latches all control signals + data values. On reset or bubble: all outputs go to 0.
EX_MEM Register	Latches ALU result, ReadData2, branch adder output, Zero/BranchSelect, RD, and control signals.
MEM_WB Register	Latches ReadData from memory and ALU_Result for writeback stage.
Forwarding Unit	Compares pipeline register RDs to current RS1/RS2. Outputs 2-bit ForwardA and ForwardB.
mux_3to1 (x2)	3-to-1 mux before ALU inputs. Selects: original value (00), MEM/WB (01), EX/MEM (10).
Hazard Detection Unit	Detects load-use hazards. Drives PC_Write, IF_ID_Write, control_mux_selector_bit.
Updated Program Counter	Added PC_Write input — when 0, PC holds its current value (stall).
Updated ALU_64_bit	Added LessThan output for BLT support. Zero now computed combinationally inside always block.

4.5 Pipelined Assembly — Bubble Sort with NOPs

Array: [7, 6, 5, 4, 3, 2, 1] — worst-case reverse-sorted input. Address spacing: 4 bytes per element (addresses 256, 260, ..., 280). Counter x8 increments by 4 (byte offset), compared against x6=28 (7 elements x 4 bytes).

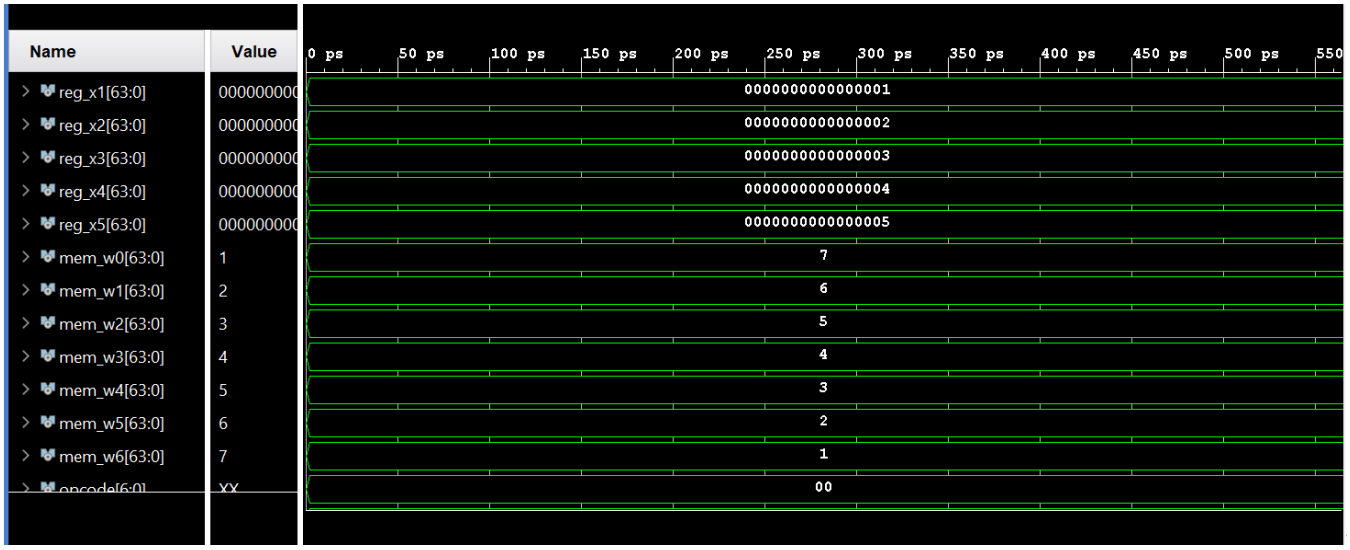
#	Hex	Assembly	Type	Purpose
1	10000293	addi x5, x0, 256	ADDI	x5 = 256 — array base address
2	00700313	addi x6, x0, 28	ADDI	x6 = 28 = 7x4 — total byte length
3	00000413	addi x8, x0, 0	ADDI	x8 = 0 — outer counter i=0
4	08640e63	bge x8, x6, END	BGE	OUTER: if i>=28 done
5-7	00000013	NOP x3	NOP	Branch delay — 3 cycles for branch to resolve
8	10000293	addi x5, x0, 256	ADDI	Reset base address for each outer pass
9	fff30493	addi x9, x6, -4	ADDI	x9 = 28-4 = 24 — inner limit base
10	408484b3	sub x9, x9, x8	SUB	x9 = inner limit - i (shrinks each pass)
11	00000513	addi x10, x0, 0	ADDI	x10 = 0 — inner counter j=0
12	06950463	bge x10, x9, OUTERINC	BGE	INNER: if j>=limit go increment outer
13-15	00000013	NOP x3	NOP	Branch delay
16	00251593	slli x11, x10, 1	SLLI	x11 = j<<1 — byte offset for arr[j]
17	00b285b3	add x11, x5, x11	ADD	x11 = base + offset = addr(arr[j])
18	00150613	addi x12, x10, 1	ADDI	x12 = j+1
19	00261613	slli x12, x12, 1	SLLI	x12 = (j+1)<<1 — byte offset for arr[j+1]
20	00c28633	add x12, x5, x12	ADD	x12 = base + offset = addr(arr[j+1])
21	0005a683	lw x13, 0(x11)	LW	x13 = arr[j] loaded from memory
22	00062703	lw x14, 0(x12)	LW	x14 = arr[j+1] loaded from memory
23	02d74063	bge x13, x14, SWAP	BGE	If arr[j] >= arr[j+1] — swap needed
24-26	00000013	NOP x3	NOP	Branch delay
27	00000c63	beq x0, x0, INNERINC	BEQ	Always jump — skip swap (no-swap path)
28-30	00000013	NOP x3	NOP	Branch delay
31	00e5a023	sw x14, 0(x11)	SW	SWAP: write arr[j+1] value to arr[j]'s address
32	00d62023	sw x13, 0(x12)	SW	SWAP: write arr[j] value to arr[j+1]'s address
33	00150513	addi x10, x10, 1	ADDI	INNERINC: j++
34	fa0004e3	beq x0, x0, INNER	BEQ	Always jump back to inner loop
35-37	00000013	NOP x3	NOP	Branch delay
38	00140413	addi x8, x8, 4	ADDI	OUTERINC: i += 4 (byte offset per pass)
39	f6000ae3	beq x0, x0, OUTER	BEQ	Always jump back to outer loop

#	Hex	Assembly	Type	Purpose
40-43	00000013	NOP x4	NOP	Branch delay + padding

5. Pipelined Waveform Analysis

5.1 Image 1 — Initial State (t = 0 to 550 ps)

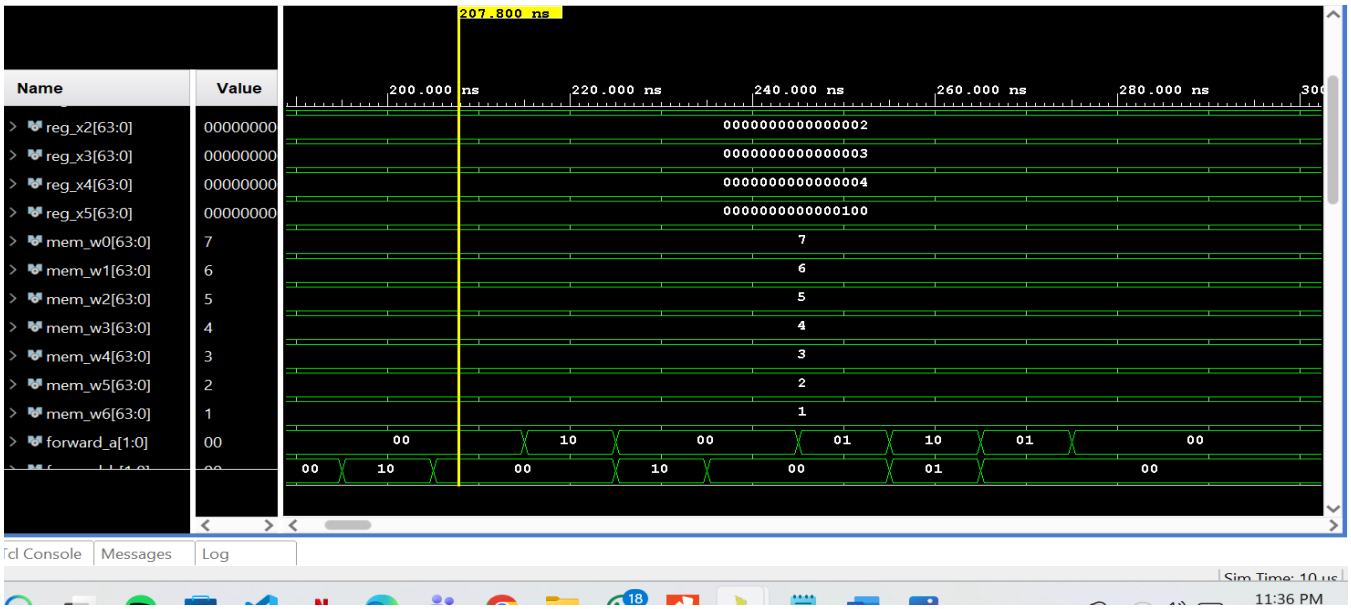
This waveform shows the beginning of simulation immediately after reset is released. The pipeline is being filled for the first time.



Signal	Value	Explanation
reg_x1 to reg_x5	1, 2, 3, 4, 5	Registers initialized to index value — x1=1, x2=2, etc.
mem_w0 to mem_w6	7, 6, 5, 4, 3, 2, 1	Initial unsorted array in data memory — worst case input
opcode	XX then 13	Transitioning from reset. 13 (hex) = 0010011 = ADDI opcode — first instruction
rs1, rs2, rd	00, 00, 05	First instruction: addi x5, x0, 256. RD=x5, RS1=x0
reg_x5	5 initially then 0x100	x5 transitions from initial value 5 to 256 (0x100) as addi executes

5.2 Image 2 — Early Execution

The pipeline is now fully operational. Instructions are flowing through all 5 stages. This waveform shows the program setup phase.

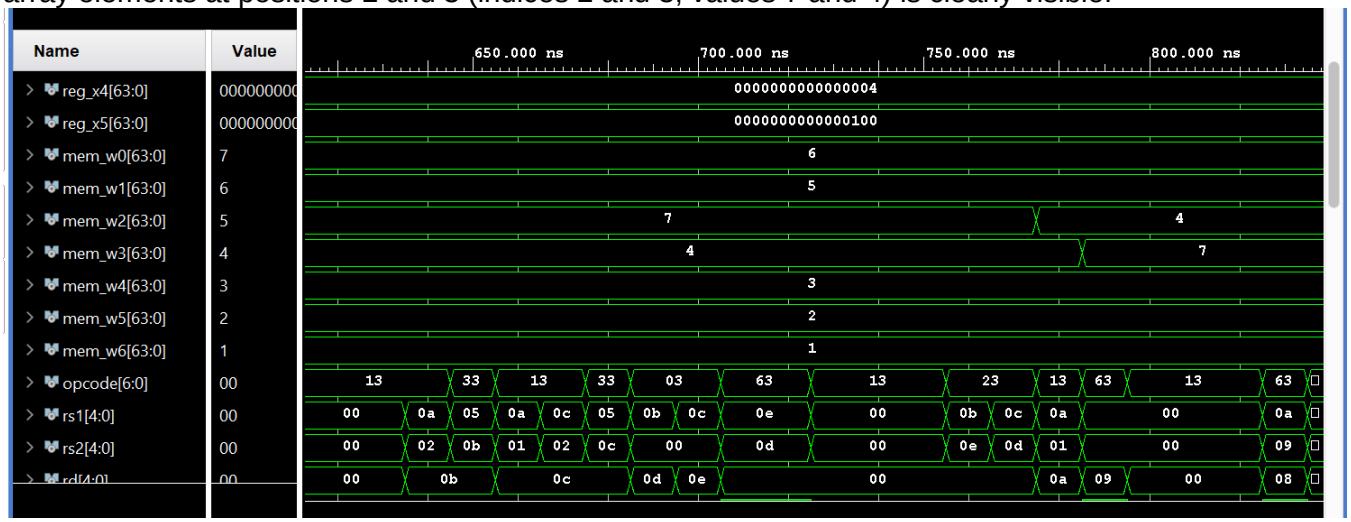


Signal	Value	Explanation
reg_x5	5 -> 0x100 (256)	x5 successfully loaded with base address 256 after addi x5,x0,256 commits
mem_w0..w6	7, 6, 5, 4, 3, 2, 1	Memory unchanged — no swaps have occurred yet in this phase
opcode sequence	00, 13, 63, 13, 33...	00=reset, 13=ADDI, 63=BRANCH, 33=R-type cycling through instructions
rs1, rs2, rd values	Changing per instruction	Each cycle a new instruction is decoded — operand fields change accordingly
reg_x4	4 throughout	x4 not yet used in setup phase — retains initial value

> **Pipeline Fill Observation**
> Notice how the opcode changes every 10ns (1 clock cycle). Instructions flow continuously: addi -> addi -> addi -> bge -> nop -> nop -> nop -> ... This is the pipeline working — one instruction per cycle moving through stages.

5.3 Image 3 — Middle of Execution: First Swaps Occurring (650 to 800 ns)

This is the most important waveform to explain — it shows bubble sort actually working. A swap of array elements at positions 2 and 3 (indices 2 and 3, values 7 and 4) is clearly visible.

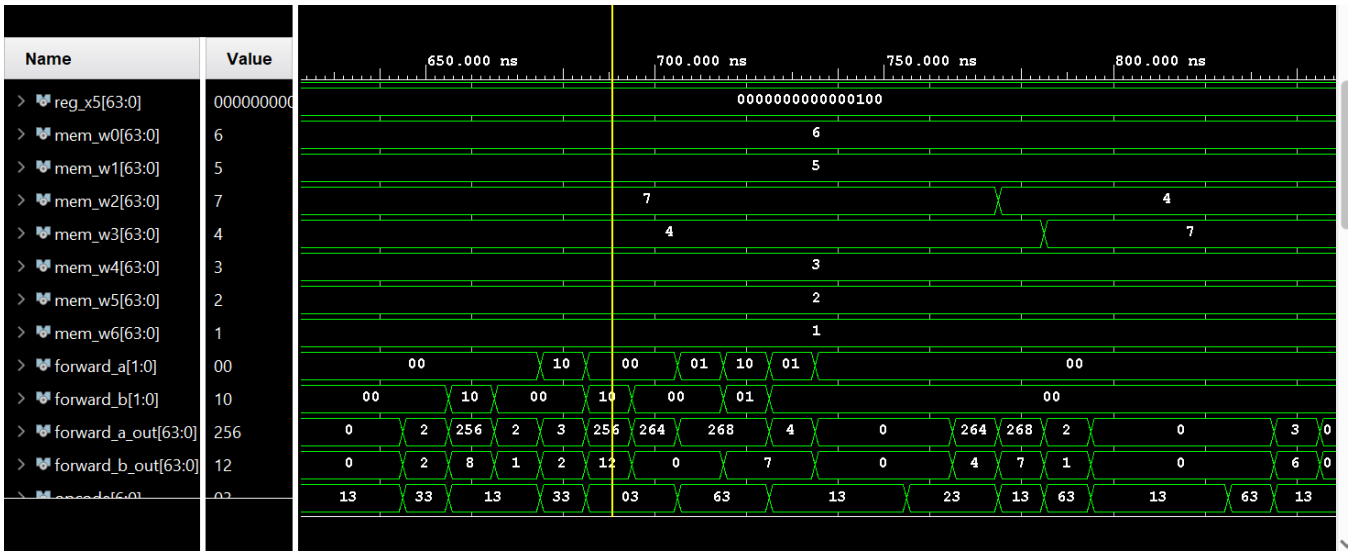


Signal	Value Transition	Explanation
mem_w2	7 -> 4	Array position 2 value changes from 7 to 4 — sw instruction writing arr[j+1] to arr[j]'s address
mem_w3	4 -> 7	Array position 3 value changes from 4 to 7 — sw instruction writing arr[j] to arr[j+1]'s address
mem_w0, mem_w1	6, 5	Earlier positions already partially sorted from first outer pass
mem_w4, mem_w5, mem_w6	3, 2, 1	Later positions untouched in this pass — sorting progresses left to right
opcode 63 then 13 then 23	Branch, NOP, Store	BGE (63) triggers branch, NOPs follow, then SW (23) executes the swap
rs1=0x0b, rs2=0x0e -> rd=0x0a	Changing	Instruction 31: sw x14,0(x11) — RS2=x14(0e), RS1=x11(0b), no RD for store

Key Observation — Swap in Action
At approximately 750ns, you can see mem_w2 drop from 7 to 4 and simultaneously mem_w3 rise from 4 to 7. This is exactly the two store instructions (sw x14,0(x11) and sw x13,0(x12)) executing. The array is being sorted in real-time.

5.4 Image 4 — Forwarding Unit Active (650 to 800 ns)

This waveform zooms into the same time period but now shows the forwarding signals, proving the hazard handling hardware is working correctly.



Signal	Value	Explanation
forward_a[1:0]	00 -> 10 -> 01 -> 00	Forwarding for first ALU operand: 10=from EX/MEM, 01=from MEM/WB, 00=no forward
forward_b[1:0]	00 -> 10 -> 01 -> 00	Forwarding for second ALU operand — same pattern, different timing
forward_a_out[63:0]	0,2,256,2,3,256,264,268...	The actual value being forwarded — address computations for arr[j] and arr[j+1]
forward_b_out[63:0]	0,2,8,1,2,12,0,7,0,4...	Second forwarded value — the loaded array element values (7,6,5...) during comparison
mem_w2 and mem_w3 swap	7->4 and 4->7	Visible simultaneously — confirms the swap completing correctly while forwarding active

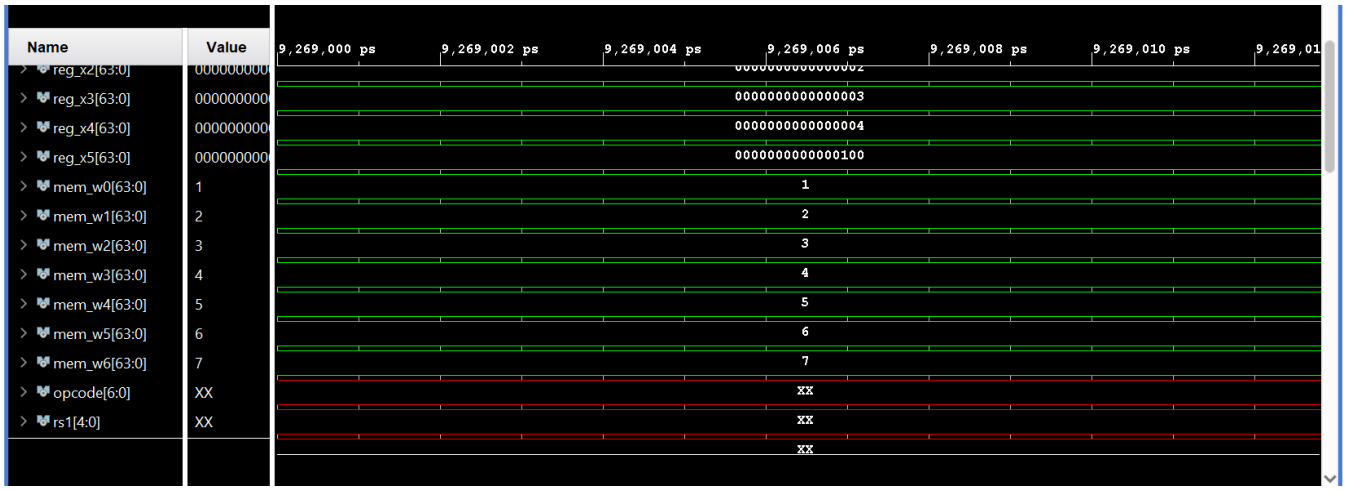
The forward_a and forward_b signals change between 00, 01, and 10 multiple times per inner loop iteration. This proves the forwarding unit is actively resolving data hazards during the address computation and comparison instructions (slli, add, lw, bge sequence).

> **Forwarding Evidence**
> forward_a transitions 00->10->01->00 within the span of a few clock cycles. This shows: no hazard (00), then EX/MEM forward needed (10 — result of previous instruction), then MEM/WB forward (01 — result from two instructions ago), then back to normal. The processor never stalls for these R-type/I-type dependencies.

5.5 Final State — Sorted Array

At the end of simulation, after all outer and inner loop passes complete, the data memory contains the fully sorted array:

mem_w0	mem_w1	mem_w2	mem_w3	mem_w4	mem_w5	mem_w6
1	2	3	4	5	6	7



Result
Pipelined Bubble Sort: SUCCESSFUL. Input [7, 6, 5, 4, 3, 2, 1] (worst-case reverse-sorted) correctly sorted to [1, 2, 3, 4, 5, 6, 7]. All three hazard types handled correctly: data hazards via forwarding, load-use via stall, control hazards via NOP insertion.

6. Key Verilog Modules

6.1 Top-Level Pipelined Processor

The top-level module instantiates and connects all components. Key connections shown:

```
// Branch mux uses EX/MEM stage signals (not current cycle)
mux branch_mux(Out1, EX_MEM_Adder_out,
               (EX_MEM_Branch & EX_MEM_Zero), PC_in);

// PC with stall support
Program_Counter PC(clk, reset, PC_Write, PC_in, PC_out);

// Forwarding 3-to-1 muxes before ALU
mux_3to1 mux_ForwardA(ID_EX_ReadData1, WriteData,
                      EX_MEM_ALU_Result, ForwardA, ForwardA_out);
mux_3to1 mux_ForwardB(ID_EX_ReadData2, WriteData,
                      EX_MEM_ALU_Result, ForwardB, ForwardB_out);
```

6.2 Forwarding Unit

```
module forwarding_unit(
    input [4:0] EX_MEM_RD, MEM_WB_RD, ID_EX_RS1, ID_EX_RS2,
    input EX_MEM_RegWrite, MEM_WB_RegWrite,
    output reg [1:0] ForwardA, ForwardB
);
always @(*) begin
    // EX/MEM forward (most recent) takes priority
    if (EX_MEM_RegWrite && EX_MEM_RD != 0 && EX_MEM_RD == ID_EX_RS1)
        ForwardA = 2'b10;
    // MEM/WB forward (only if EX/MEM not already forwarding)
    else if (MEM_WB_RegWrite && MEM_WB_RD != 0 && MEM_WB_RD == ID_EX_RS1
             && !(EX_MEM_RegWrite && EX_MEM_RD != 0 && EX_MEM_RD == ID_EX_RS1))
        ForwardA = 2'b01;
    else
        ForwardA = 2'b00;  // No forwarding
    // Same logic for ForwardB using ID_EX_RS2
end
endmodule
```

6.3 Hazard Detection Unit

```
module Hazard_detection_unit(
    input [4:0] RS1, RS2, ID_EX_RD,
    input ID_EX_MemRead,
    output reg IF_ID_write, PC_Write, mux_selector_bit
);
always @(*) begin
    if (ID_EX_MemRead && (ID_EX_RD == RS1 || ID_EX_RD == RS2)) begin
        mux_selector_bit = 1;  // Inject bubble into ID/EX
        IF_ID_write = 0;       // Freeze IF/ID register
        PC_Write = 0;          // Freeze PC
    end else begin
```

```
        mux_selector_bit = 0;
        IF_ID_write = 1;
        PC_Write = 1;
    end
end
endmodule
```

7. Conclusion

This assignment successfully implemented Bubble Sort on both a single-cycle and a 5-stage pipelined 64-bit RISC-V processor in Verilog.

- The single-cycle processor correctly sorted [44, 88, 79, 3, 79, 1, 8] to [1, 3, 8, 44, 79, 79, 88] using 25 instructions with a straightforward datapath design.
- The pipelined processor correctly sorted [7, 6, 5, 4, 3, 2, 1] to [1, 2, 3, 4, 5, 6, 7] — the worst-case reverse-sorted input — using 43 instructions (including NOPs for branch delay slots).
- Data hazards between consecutive R-type and I-type instructions are resolved with zero performance penalty using the Forwarding Unit, which routes values directly from EX/MEM or MEM/WB pipeline registers to the ALU input.
- Load-use hazards are resolved by the Hazard Detection Unit, which stalls the pipeline for one cycle by freezing the PC and IF/ID register while injecting a bubble into ID/EX.
- Control hazards from branches are resolved in software by inserting 3 NOP instructions after every branch, eliminating the need for hardware branch prediction or flushing.
- Waveform analysis in Vivado confirmed all operations: the swap of mem_w2 and mem_w3 (7<->4) is directly visible at ~750ns, and the forwarding signals (forward_a, forward_b) switching between 00, 01, and 10 confirm active hazard resolution.

The pipelined implementation demonstrates the fundamental tradeoff in processor design: higher throughput at the cost of additional hardware complexity and the need for hazard management.